9 АРХИТЕКТУРА ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ И ПРИМЕНЕНИЕ ПАТТЕРНОВ ПРОЕКТИРОВАНИЯ

9.1 Архитектурные принципы и уточнение платформенной среды

Настоящая глава дополняет спецификацию требований к программному обеспечению (SRS) проекта «Персональный тайм-трекер на Java». В ней фиксируются архитектурные решения, обоснование выбранных паттернов и способ организации сборки. Выбор выполнен с учётом контекста проекта и рекомендаций лекции № 3: паттерн должен решать конкретную проблему, а его применение не должно создавать избыточную сложность.

Для приложения выбрана слоистая архитектура. Представление (MainWindow и MiniTimerWindow) отвечает за взаимодействие с пользователем. Прикладной слой содержит сценарии работы с задачами и таймером. Доменная модель описывает категории, задачи, рабочие сеансы и паузы, а интерфейсы репозиториев отделяют правила приложения от конкретного способа хранения. Инфраструктурный слой содержит реализации SQLite и адаптер интеграции с рабочим столом.

Уточнение по замечанию преподавателя. В первоначальной редакции SRS Windows была указана как основная целевая операционная система. Это решение не следует считать окончательным ограничением. В дальнейшей разработке целевая ОС не фиксируется заранее: совместимость с Windows, Linux и macOS проверяется отдельно. Основная логика не должна зависеть от наличия системного трея; если конкретная графическая среда его не поддерживает, используется резервная реализация NoOpDesktopIntegration, а основное окно остаётся доступным.



Рисунок 6 — Слоистая архитектура и границы компонентов приложения

9.2 Выбор и обоснование паттернов проектирования

Паттерны отбирались по наличию конкретной задачи в проекте, а не по количеству. В результате выбраны три поведенческих паттерна, один структурный и один порождающий. Такой набор покрывает управление состояниями таймера, рассылку обновлений, заменяемые алгоритмы аналитики, интеграцию с API рабочего стола и создание форматов экспорта.

Паттерн и категория	Проблема проекта	Классы реализации
State — поведенческий	Набор допустимых действий зависит от состояния таймера.	TimerService, TimerState, IdleState, RunningState, PausedState, FinishedState.
Observer — поведенческий	Несколько элементов интерфейса должны получать одинаковые обновления таймера.	TimerService, TimerObserver, TimerSnapshot, MainWindow, MiniTimerWindow.
Strategy — поведенческий	Правила выдачи рекомендаций могут изменяться и комбинироваться.	RecommendationService, RecommendationStrategy, OvertimeRecommendationStrategy, FrequentPauseRecommendationStrategy.
Adapter — структурный	Прикладной код не должен зависеть от конкретного API системного трея.	DesktopIntegration, AwtDesktopIntegrationAdapter, NoOpDesktopIntegration.
Factory Method — порождающий	Создание PDF- и CSV-экспортёров должно выбираться по формату отчёта.	ReportExporterFactory, CsvReportExporterFactory, PdfReportExporterFactory, ReportExporter.

9.3.1 Паттерн State (Состояние)

Контекст — TimerService. Интерфейс TimerState задаёт операции start, pause, resume и stop. Конкретные состояния IdleState, RunningState, PausedState и FinishedState определяют, какие действия допустимы в данный момент. TimerService делегирует операцию текущему объекту состояния, а после успешного действия заменяет его следующим состоянием.

Пример взаимодействия: после запуска задачи TimerService переходит в RunningState. Вызов pause(reason) передаётся текущему состоянию; RunningState разрешает операцию и инициирует сохранение причины паузы, после чего устанавливается PausedState. Вызов resume() в PausedState закрывает запись о паузе и возвращает таймер в RunningState. Попытка выполнить недопустимую операцию приводит к IllegalStateException. Это исключает разрастание одного метода множеством условных операторов.

9.3.2 Паттерн Observer (Наблюдатель)

TimerService выступает источником событий, а MainWindow и MiniTimerWindow — подписчиками. Контракт TimerObserver содержит метод onTimerUpdated(TimerSnapshot). При старте, паузе, продолжении,

остановке и периодическом обновлении TimerService формирует неизменяемый снимок состояния и передаёт его всем подписчикам.

Список наблюдателей хранится в CopyOnWriteArrayList, поэтому изменение подписок не требует ручного управления синхронизацией при обходе коллекции. Исключение, возникшее в одном представлении, не должно прекращать уведомление остальных. Графический интерфейс переносит обновление компонентов на поток Swing через SwingUtilities.invokeLater. Благодаря этому таймер не зависит от количества окон, которые отображают его состояние.

9.3.3 Паттерн Strategy (Стратегия)

RecommendationService хранит список объектов RecommendationStrategy и последовательно вызывает evaluate(AnalyticsInput). Каждая стратегия анализирует свой критерий и возвращает Optional<Recommendation>: отсутствие рекомендации означает, что соответствующее условие не выполнено.

В текущем прототипе реализованы OvertimeRecommendationStrategy (сравнение фактической и плановой длительности с заданным порогом) и FrequentPauseRecommendationStrategy (анализ количества пауз). Новое правило можно добавить отдельным классом, реализующим RecommendationStrategy, не изменяя алгоритм RecommendationService. Числовые пороги остаются проектными параметрами и не выдаются за подтверждённые ответы заказчика.

Преимущество решения — независимое развитие правил аналитики и возможность проверять каждую стратегию отдельно. Ограничение — набор стратегий должен быть настроен при создании сервиса; пользовательское редактирование правил не входит в текущую промежуточную версию.

9.3.4 Паттерн Adapter (Адаптер)

Целевой интерфейс DesktopIntegration изолирует прикладную часть от API рабочего стола. AwtDesktopIntegrationAdapter реализует его средствами

AWT: проверяет доступность SystemTray, добавляет TrayIcon, отображает уведомления и предоставляет команды открытия или завершения приложения. DesktopIntegrationFactory выбирает адаптер только при наличии графической среды и поддержке системного трея.

Если системный трей недоступен или среда работает в headless-режиме, выбирается NoOpDesktopIntegration. Резервная реализация не выполняет платформенных действий, но сохраняет единый контракт. Это позволяет не привязывать основную логику к Windows и постепенно проверять поведение на других ОС.

9.3.5 Паттерн Factory Method (Фабричный метод)

ReportExporterFactory является абстрактным создателем и объявляет фабричный метод createExporter(), возвращающий интерфейс ReportExporter. Конкретные создатели CsvReportExporterFactory и PdfReportExporterFactory возвращают CsvReportExporter и PdfReportExporter соответственно. ReportService выбирает конкретную фабрику по значению ExportFormat и вызывает общий метод export().

Клиентский код зависит от интерфейса ReportExporter, а не от конкретной библиотеки или формата. Добавление нового формата потребует нового экспортёра и конкретной фабрики; существующий интерфейс сервиса при этом сохраняется.



Рисунок 7 — Участники пяти выбранных паттернов проектирования

10 СИСТЕМА СБОРКИ И ОРГАНИЗАЦИЯ JAVA-ПРОЕКТА

10.1 Выбор Maven

Для сборки выбран Apache Maven. В лекции № 4 Maven описан как декларативный инструмент: в pom.xml задаются координаты проекта, версия Java, зависимости и плагины, а выполнение организуется через стандартный жизненный цикл. Для учебного проекта это обеспечивает предсказуемую структуру и воспроизводимую сборку.

Система	Особенность	Оценка для проекта
Ant	Императивное описание шагов в build.xml; зависимости и classpath часто приходится настраивать вручную.	Не выбран: для текущего проекта потребовал бы больше ручной конфигурации.
Maven	Декларативный pom.xml, стандартные каталоги, жизненный цикл и управление транзитивными зависимостями.	Выбран: соответствует методичке и упрощает сборку, подключение библиотек и упаковку JAR.
Gradle	Гибкий DSL на Groovy/Kotlin, граф задач и развитая инкрементальная сборка.	Возможен, но для проекта не даёт критичного преимущества перед Maven.

10.2 Координаты и конфигурация pom.xml

Параметр	Значение
groupId	ru.mirea.khusyainov
artifactId	personal-time-tracker
version	1.0.0
packaging	jar
Версия компиляции	Java 17 (maven.compiler.release=17)
Главный класс	ru.mirea.khusyainov.timetracker.App
Итоговый файл	target/personal-time-tracker.jar

В pom.xml настроены maven-compiler-plugin 3.13.0, maven-surefire-plugin 3.2.5 и maven-shade-plugin 3.5.3. Последний формирует исполняемый JAR с runtime-зависимостями и записывает главный класс в манифест. Используется ServicesResourceTransformer для сохранения записей SPI подключённых библиотек.

10.3 Зависимости, структура и проверка сборки

Зависимость	Версия / область	Назначение
sqlite-jdbc	3.46.1.0, runtime	Локальная база SQLite через JDBC.
OpenPDF	1.3.39, runtime	Формирование PDF-отчётов.
JUnit Jupiter	5.10.2, test	Подготовленная тестовая зависимость для практической работы № 4; в runtime JAR не включается.

Исходный код содержит 44 Java-файла и разделён на пакеты domain.model, domain.repository, application.task, application.timer, application.analytics, application.report, infrastructure.persistence, infrastructure.platform и presentation. Ресурсы находятся в src/main/resources; в том числе включён шрифт DejaVu Sans с лицензией для одинакового отображения кириллицы при экспорте PDF на разных ОС. Каталог target создаётся Maven и не хранится в Git.

README.md содержит назначение приложения, используемые библиотеки, структуру каталогов, предварительные условия, команду сборки и пример запуска. Для сборки используется команда:

```
mvn clean package
```

После успешной сборки исполняемый файл запускается командой:

```
java -jar target/personal-time-tracker.jar
```

Для автоматизированной проверки собранного JAR без открытия графического окна предусмотрен диагностический режим:

```
java -jar target/personal-time-tracker.jar --self-check
```

Самопроверка создаёт временную базу SQLite, проверяет сохранение сеанса и причины паузы, выполнение стратегий рекомендаций и создание CSV/PDF-файлов. В GitHub Actions на коммите fda8319 успешно завершились сборка и самопроверка на Ubuntu, Windows и macOS: mvn clean package и запуск JAR в режиме --self-check. Это подтверждает переносимость сборки и базовых интеграций; фактическое поведение GUI и системного трея на каждой ОС дополнительно проверяется в соответствующей графической среде.

Список использованных источников

1. Яковлев М. С. Лекция № 3 «Паттерны проектирования». — МИРЭА — Российский технологический университет, 2026.
2. Яковлев М. С. Лекция № 4 «Системы сборки». — МИРЭА — Российский технологический университет, 2026.
3. Практическое занятие № 3 «Паттерны проектирования. Системы сборки» по дисциплине «Проектирование и архитектура программного обеспечения». — 2026.
4. Хусяинов Э. Р. Спецификация требований к программному обеспечению «Персональный тайм-трекер на Java». Практическое занятие № 2. — 2026.